

SilkSphere Technical Documentation

Comprehensive documentation covering the IoT architecture, AI model integration, marketplace workflows, API endpoints, and database schema for the SilkSphere precision silkworm farming platform.

Week 10 Deliverable | Version 1.0 | November 2026 | ESP32 + FastAPI + React + Firebase

4	9	2	4	8
System Layers	API Endpoints	AI Models	Firestore Collections	Lifecycle Stages



DOCUMENT 01

System Architecture

IoT hardware design, AI pipeline diagrams, database architecture, technology stack, and known issues.

- 4-layer system overview diagram
- ESP32 hardware wiring schematic
- Two-stage YOLO + Keras AI pipeline
- Environmental threshold tables
- Closed-loop fan automation flow
- Firestore + flat-file database design
- Full technology stack reference



DOCUMENT 02

User Manual

Step-by-step guide for farmers and buyers — every feature, all workflows, and quick-reference tables.

- Getting started & demo credentials
- Farmer: Dashboard, Lifecycle, Batches
- Farmer: AI Diagnostics (upload + live cam)
- Farmer: Environment History, Analytics
- Buyer: Marketplace & Silk Passport
- Buyer: Cart, Orders, Wallet
- Grade system & alert type reference
- Troubleshooting guide



DOCUMENT 03

API & Database Reference

All 9 REST endpoints with full request/response schemas, Firestore collections, flat-file storage, and localStorage structure.

- All 9 endpoints documented in full
- Request body field tables (type, required, default)
- Response field descriptions
- Rate limit and error response reference
- Firestore collection schemas
- sensor_log.txt CSV column spec
- localStorage key reference
- ESP32 firmware data contract

What's Covered

ESP32 IoT Node
DHT11/22, buzzer, CO_2 placeholder, WiFi HTTP, firmware logic

YOLOv8 Detection
Object detection, NMS, SAHI sliced inference for dense trays

MobileNetV2 Classifier
Per-crop Keras classification, confidence fusion (40/60 weights)

Closed-Loop Automation
Automatic fan control based on sensor thresholds by instar stage

Marketplace Workflow
Silk Passport, batch grading, procurement cart, order flow

Firebase Firestore
sensor_data, device_commands, sensor_history — realtime listeners

REST API
9 endpoints — disease prediction, sensor ingestion, motor control, forecasting

Lifecycle Prediction
Rule-based stage forecasting, env compliance scoring, cocoon grade formula

Farmer Features
Dashboard, Batches, Lifecycle, Diagnostics, Analytics, Wallet

Buyer Features
Marketplace, Silk Passport, cart, procurement, orders

SilkSphere · Technical Documentation · Week 10 Deliverable · Version 1.0 · November 2026

IoT Architecture · AI Model Integration · Marketplace Workflows · API Reference · Database Schema

⚙️ System Architecture

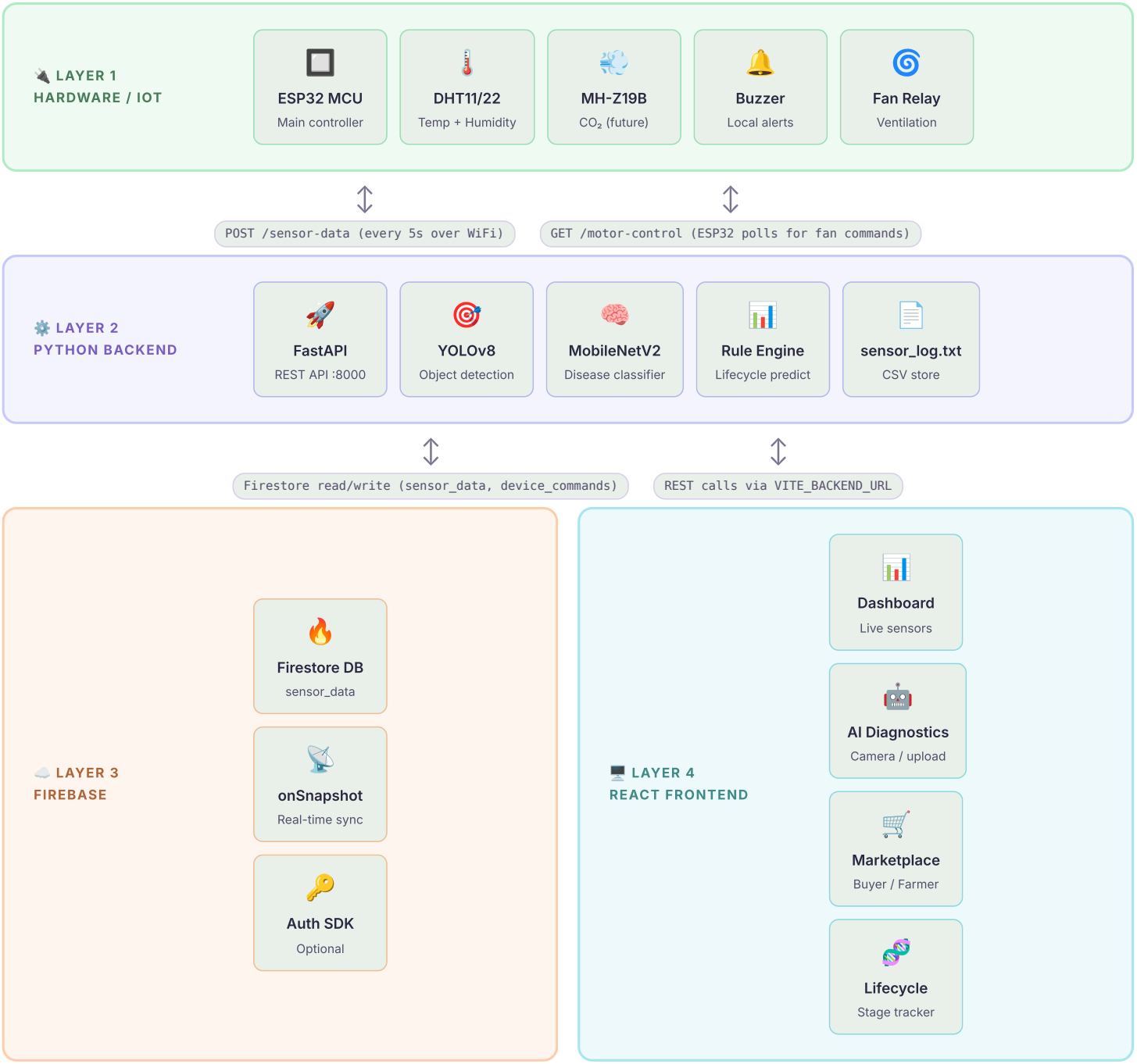
IoT hardware, AI model integration, cloud database design, and technology stack

Week 10 — Technical Documentation | Version 1.0 | ESP32 + FastAPI + React | YOLOv8 + MobileNetV2

CONTENTS

1. System Overview
2. IoT Architecture
3. AI Model Pipeline
4. Database Architecture
5. Technology Stack
6. Known Issues

🏗️ 1. System Overview



🔌 2. IoT Architecture

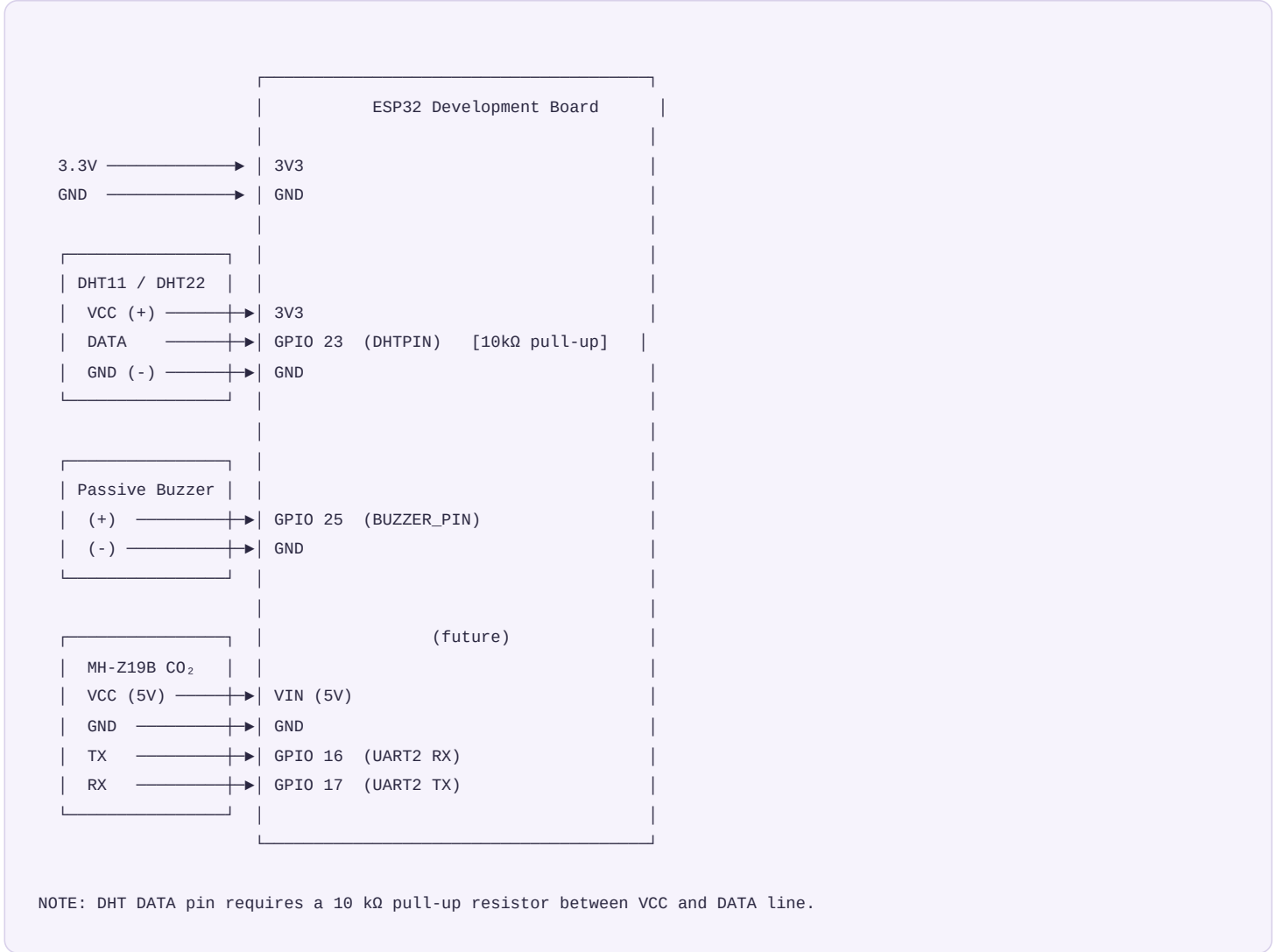
2.1 Hardware Components

2.2 Firmware Data Flow

1. **Power On**
   Initialize DHT sensor, buzzer. Play startup beep sequence.
2. **Connect WiFi**
   Attempts connection to hardcoded SSID. 3 quick beeps on success, 1 long beep on failure → offline mode.
3. **Every 5 seconds**
   Read DHT → check thresholds → beep if alert → HTTP POST to `/sensor-data`.
4. **Offline mode**
   If WiFi fails, readings are logged to Serial only. No data is lost permanently.

COMPONENT	MODEL	GPIO	PURPOSE
Microcontroller	`ESP32`	—	WiFi-enabled MCU, runs all firmware logic
Temp + Humidity	`DHT11/DHT22`	**GPIO 23**	Reads °C and relative humidity %
Audible Alert	Passive Buzzer	**GPIO 25**	Local threshold alert tones
CO_2 Sensor *(future)*	`MH-Z19B`	GPIO 16/17	UART2 serial; placeholder 400 ppm now

2.3 Hardware Wiring Schematic

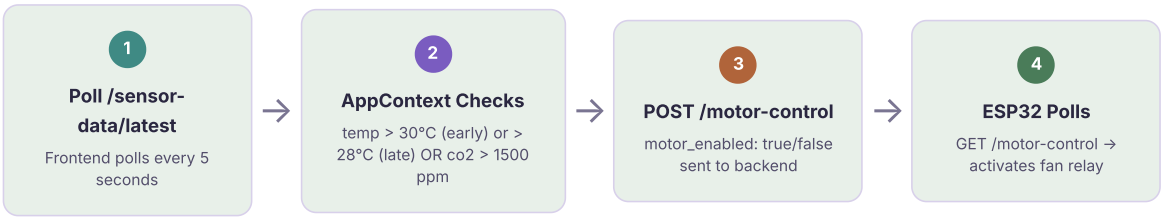


NOTE: DHT DATA pin requires a 10 kΩ pull-up resistor between VCC and DATA line.

2.4 Environmental Threshold Table

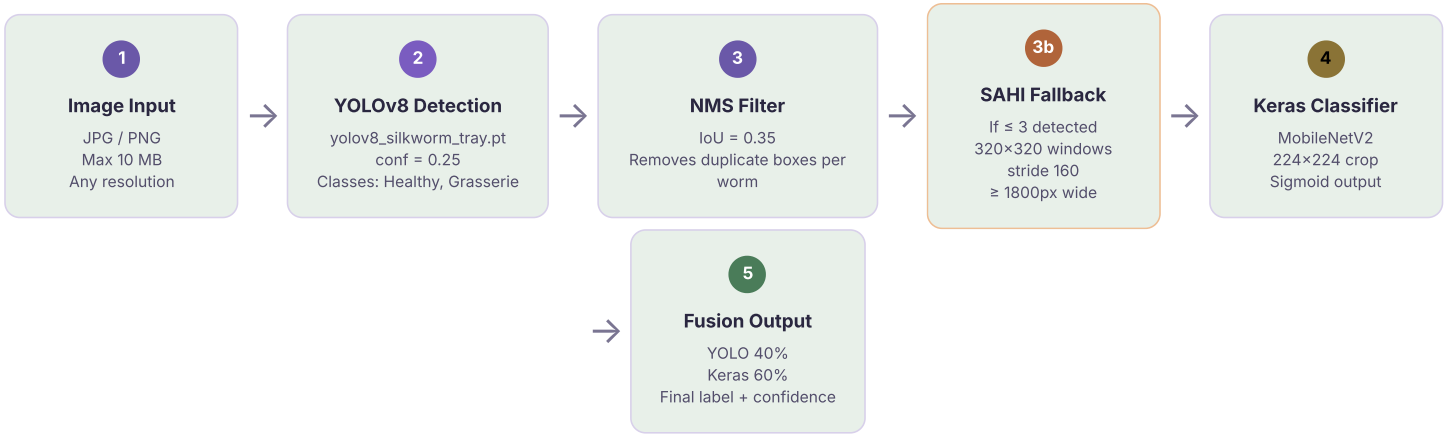
STAGE	IDEAL TEMP	ALERT TEMP	IDEAL HUMIDITY	ALERT HUMIDITY	CO_2 ALERT
Instar I–III	26–28 °C	< 24 or > 30 °C	80–85 %	< 70 or > 90 %	> 1500 ppm
Instar IV–V	22–26 °C	< 20 or > 28 °C	70–80 %	< 55 or > 90 %	> 1500 ppm
Cocoon / Pupal	23–25 °C	< 20 or > 28 °C	65–75 %	< 60 or > 85 %	> 1500 ppm

2.5 Closed-Loop Fan Automation



🤖 3. AI Model Architecture

3.1 Two-Stage Disease Detection Pipeline



3.2 Fusion Logic

✅ Both Agree

Both YOLO and Keras predict the same class

```
final_conf = 0.4 × YOLO_conf + 0.6 × Keras_conf
```

⚠️ Disagree

Models predict different classes — weighted vote

```
grasserie_score = 0.4×YOLO + 0.6×Keras
winner = Grasserie if score ≥ 0.5
```

⏭️ Keras Skipped

Crop too small (< 8px) or model not loaded

```
Keep YOLO result unchanged
```

3.3 Model Files

FILE	ARCHITECTURE	INPUT	OUTPUT	ROLE
`yolov8_silkworm_tray.pt`	YOLOv8s	Any size image	Bounding boxes + class	Primary detector (tray-optimised)
`yolov8_silkworm.pt`	YOLOv8s	Any size image	Bounding boxes + class	Fallback detector
`silkworm_disease_model.keras`	MobileNetV2 + Dense head	224×224×3 RGB	Sigmoid scalar 0–1	Per-crop classifier (secondary)

3.4 Lifecycle Prediction Model (Rule-Based)

STAGE	IDEAL TEMP	IDEAL HUMIDITY	IDEAL CO_2	DURATION
Egg	27 °C	82 %	350 ppm	10 days
Instar 1–3	27 °C	82 %	350 ppm	3 days each
Instar 4	24 °C	75 %	350 ppm	4 days
Instar 5	24 °C	75 %	350 ppm	5 days
Spinning	24 °C	70 %	350 ppm	4 days
Cocoon	24 °C	70 %	350 ppm	3 days

ℹ️ **Grade formula:** `overall = (env_score × 0.4) + (ai_health × 0.6)` → A ≥ 85, B ≥ 70, C ≥ 50, D < 50. Grade drops by 1 if temp > 31°C, temp < 19°C, humidity < 55%, or CO_2 > 1300 ppm.

🗄️ 4. Database Architecture

COLLECTION / PATH	FIELDS	WRITTEN BY	READ BY
`sensor_data/latest`	temperature, humidity, co2, light, nodeStatus, timestamp, zones	AppContext (on real ESP32 data)	AppContext onSnapshot listener
`device_commands/status`	fanState, manualOverride, timestamp	AppContext (manual + auto)	AppContext onSnapshot, ESP32 (future)
`device_commands` (collection)	command (FAN_ON/OFF), type (manual/auto), timestamp	AppContext	Audit trail
`sensor_history` (collection)	temperature, humidity, co2, timestamp	Planned (not yet implemented)	updatePredictions(), Simulation page
`sensor_log.txt` (flat file)	datetime, node_id, temp, humidity, co2 (CSV)	FastAPI /sensor-data	FastAPI /sensor-data/history, /latest

🛠️ 5. Technology Stack

🖥️ Frontend

Framework	`React 19`
Build	`Vite 8`
Charts	`Recharts 3`
Icons	`Lucide React`
Deploy	`Vercel`

⚙️ Backend

Framework	`FastAPI 0.162`
Server	`Uvicorn 0.54`
Rate limit	`slowapi 0.1.10`
Detection	`YOLOv8 (ultralytics)`
Classifier	`TensorFlow 2.21`
Vision	`OpenCV 5.0`

☁️ Cloud + IoT

Database	`Firebase Firestore`
Auth SDK	`Firebase Admin 7.7`
MCU	`ESP32`
Sensor	`DHT11/DHT22`
Firmware	`Arduino C++`

⚠️ 6. Known Issues & Limitations

⚠️ **Motor control URL hardcoded:** AppContext.jsx uses `http://localhost:5000/motor-control` instead of the VITE_BACKEND_URL. Fan control will fail in deployed environments. Fix: replace with `BACKEND_URL + '/motor-control'`.

⚠️ **sensor_log.txt grows indefinitely:** No log rotation implemented. For production, migrate to Firestore `sensor_history` or add a cron job to rotate the file.

⚠️ **Firebase Auth unused:** `firebase/auth` is imported but login/register are localStorage-only. Implement Firebase Auth for production security.

⚠️ **CO_2 always 400 ppm:** ESP32 firmware sends a hardcoded value. The MH-Z19B sensor is wired but not read yet. Add UART2 parsing to get real CO_2 data.

ℹ️ **Blockchain is cosmetic:** The hash shown in Silk Passport (`0x4f3a8b...`) is a static string. No actual blockchain integration exists. A real hash function (SHA-256 of batch data) should be implemented for genuine traceability.

📖 User Manual

Complete guide for silkworm farmers and cocoon buyers — getting started, all features, and reference tables

Farmer Guide | Buyer Guide | Week 10 — Version 1.0

🚀 Getting Started

Accessing the Application

Open your browser and go to your SilkSphere URL. During development this is `http://localhost:5173`. In production use your Vercel deployment URL.

Demo Credentials

🌾

Farmer Account

Email	`farmer@silk.com`
Password	`demo123`

🛒

Buyer Account

Email	`buyer@silk.com`
Password	`demo123`

Creating a New Account

1. **Click "Create account"** on the login screen
2. **Fill in your name, email, and password**
3. **Select your role** — Farmer (enter farm name) or Buyer (enter company name)
4. **Click "Create Account"** — you are logged in immediately

🌾 Part A — Farmer Guide

A1. Dashboard

Your home screen. Gives a live view of your rearing house environment and the health of your active batch.

Live Environment Cards

Temperature	Humidity	CO_2	Light
24–28 °C	70–85 %	< 1500 ppm	Day/Night cycle
Updated every 5 s	Updated every 5 s	Updated every 5 s	Updated every 5 s

Each card shows a 30-point rolling sparkline so you can spot trends at a glance.

NODE STATUS	MEANING
`Online - Real Hardware`	ESP32 is connected and sending live data
`Simulated - Smart Mock Data`	Backend is not receiving hardware data; values are computer-generated

Batch Health Summary

METRIC	DESCRIPTION
AI Health Score	Percentage from the last AI scan. 100% = all worms healthy.
Env Compliance Score	How closely your environment matches the ideal for the current instar stage.
Predicted Cocoon Grade	System forecast grade (A/B/C/D) based on combined env + AI scores.
Days to Harvest	Estimated days remaining until cocoon harvest.

Fan / Ventilation Control

💡 **Auto mode (default):** Fan turns ON automatically when temperature exceeds the stage threshold (30°C early instars, 28°C late) OR CO_2 exceeds 1500 ppm. A toast notification appears whenever the state changes.

ℹ️ **Manual Override:** Toggle the switch to take manual control, then use the Fan ON / Fan OFF button to override the automatic system.

A2. Lifecycle Tracker

Shows all 8 stages of the silkworm lifecycle with real-time progress tracking.

🥚 Egg → 🐛 Instar 1 → 🐛 Instar 2 → 🐛 Instar 3 → 🐛 Instar 4 → 🐛 Instar 5 → 🌀 Spinning → 🫘 Cocoon

Click any stage to expand and see: ideal metrics (temperature, humidity, CO_2, mortality rate) and the event diary — a timestamped log of everything that happened during that stage.

A3. My Batches

Creating a New Batch

1. **Click "+ New Batch"**
2. **Enter stage, quantity (kg), and farm name**
3. **Click "Create Batch"**
   — a unique ID is assigned (e.g., `SS-2026-0099`)

Grading a Batch

1. **Click "Grade"**
   on the batch card
2. **Select grade A, B, C, or D**
3. **Confirm**
   — batch status changes to `listed` and appears in the marketplace

ℹ️ **Batch Traceability:** Click "View Traceability" to open the full provenance report — farmer details, origin, dates, compliance scores, event timeline, and blockchain hash.

A4. AI Diagnostics

Scan your silkworms for disease using either a photo upload or live camera feed.

📷 Photo Upload Mode

1. **Click "Upload Photo"**
   or drag an image
2. **Accepts JPG / PNG, max 10 MB**
3. **Wait 2–5 s**
   for AI analysis
4. **View annotated image**
   with coloured boxes

📹 Live Camera Mode

1. **Click "Start Live Detection"**
2. **Allow camera access**
   in browser
3. **System scans every 500 ms**
   , overlays boxes live
4. **Click "Stop"**
   to end session

Reading Detection Results

BOX COLOUR	MEANING
Green	Healthy worm
Red	Grasserie (Nuclear Polyhedrosis Virus) detected

RESULT	RECOMMENDED ACTION
Healthy (95–100%)	Continue normal monitoring
Healthy (70–94%)	Re-scan in 24 hours
Grasserie Detected	Isolate infected trays; consult your extension officer

⚠️ **Backend required:** AI features only work when the Python backend is running at `http://localhost:8000`. A green indicator at the top of the Diagnostics page confirms the backend is online.

A5. Environment History

View historical charts for all 4 sensors over different time windows: **24 Hours**, **7 Days**, **30 Days**.

Each chart includes a green reference band for the healthy range, anomaly markers for threshold spikes, and stat cards showing min / max / average for the period.

A6. Farmer Marketplace

Shows your listed batches and incoming purchase orders from buyers. This is a **read-only** view — orders are confirmed automatically when buyers place them.

Each batch card shows: grade badge, price/kg, quantity, environmental compliance score, AI health score, and 7-day vitality trend.

A7. Analytics



Revenue Trend
6-month revenue area chart



Yield by Grade
Monthly A/B/C breakdown stacked bar



Env vs Ideal
30-day actual vs ideal line chart



Grade Distribution
Pie chart of overall grade mix

A8. Wallet

Adding Money

1. **Click "Add Money"**
2. **Enter amount**
3. **Select payment method**
   (GPay, PhonePe, Debit, Net Banking)
4. **Confirm**
   — balance updates immediately

Withdrawing Money

1. **Click "Withdraw"**
2. **Enter amount**
3. **Select destination**
   (bank account or UPI)
4. **Confirm**

🛒 Part B — Buyer Guide

B1. Marketplace

✅ **Verification banner:** Every listed batch has been verified by IoT sensors and AI diagnostics before appearing in the marketplace.

Filtering & Searching

FILTER	OPTIONS
Search	By Batch ID or farmer name
Grade	All / Grade A / Grade B / Grade C
Sort	By Grade, By Price (low→high), By Environmental Score

Reading a Batch Card

ELEMENT	WHAT IT SHOWS
Batch ID (e.g., `SS-2026-0042`)	Unique identifier for full traceability
Grade badge	A (gold border), B, or C
Environmental Score	% compliance with ideal rearing conditions
AI Health Score	% from automated disease scanning
7-day vitality chart	Trend of batch health over the past week
Price / Quantity	Price per kg and available quantity in kg

Placing an Order

Method 1 — From the card

Click **Place Order** → batch is added to Procurement Cart

Method 2 — Via Silk Passport

Open Silk Passport → click **Place Purchase Order**

B2. Silk Passport

Click 🛡️ **Silk Passport** on any batch card to open the full traceability report.



Scores

Env + AI health with progress bars



Blockchain Hash

Tamper-proof batch fingerprint



Env Stability

14-day chart vs ideal conditions



Daily Diary

Full event log from egg to cocoon

B3. My Orders

Shows all orders you have placed: Order ID, batch ID, farmer, quantity, total amount, date, and status (`pending` / `confirmed`).

B4. Wallet

Works the same as the Farmer Wallet — see A8. Wallet above.

📋 Reference

Grade System

GRADE	SCORE CRITERIA	TYPICAL USE
⭐ **Grade A**	Overall score ≥ 85 (env×0.4 + ai×0.6)	Premium silk yarn, export quality
🔵 **Grade B**	Overall score ≥ 70	Standard commercial silk
🟠 **Grade C**	Overall score ≥ 50	Lower-grade fabric, blends
🔴 **Grade D**	Overall score < 50	Not recommended for sale

⚠️ Grade drops by one level if the latest sensor reading shows: temperature > 31°C or < 19°C, humidity < 55%, or CO_2 > 1300 ppm.

Alert Types

COLOUR	MEANING	EXAMPLES
🔴 Red / Warning	Immediate action required	CO_2 exceeded threshold, disease detected, severe anomaly
🟡 Orange	Attention needed soon	Temperature slightly outside ideal, humidity drift
🔵 Blue / Info	Informational event	AI scan completed, batch stage changed, feeding reminder
🟢 Green / Success	All clear	Environment within ideal range, molt completed

Troubleshooting

PROBLEM	LIKELY CAUSE	FIX
AI Diagnostics page shows backend offline	Python backend not running	Run `uvicorn main:app --reload` in the `backend/` folder
Sensor shows "Simulated" data	ESP32 not connected or not sending data within 30 s	Check ESP32 WiFi config and backend URL in firmware
Fan control not working	Motor control URL hardcoded to wrong port	Ensure backend runs on port 8000 (not 5000)
CO_2 always shows 400 ppm	MH-Z19B sensor not yet integrated in firmware	Add UART2 parsing to ESP32 firmware

ℹ️ **Backend status:** Visit `http://localhost:8000/debug` in your browser to check which AI models are loaded and confirm the pipeline is active.

🔌 API & Database Reference

All 9 REST endpoints, request/response schemas, Firestore collections, flat-file storage, and localStorage structure

Base URL: http://localhost:8000 | 9 Endpoints | FastAPI + Firestore | Week 10 — Version 1.0

📡 API Endpoints

ℹ️ All endpoints return **JSON**. CORS is open (`allow_origins: ["*"]`) — restrict in production. Errors use format: `{"detail": "message"}`. File uploads use `multipart/form-data`.

GET / — Health check

RESPONSE

```
{
  "status": "Secure API is running",
  "model_loaded": true
}
```

FIELD	TYPE	DESCRIPTION
`status`	`string`	Static confirmation message
`model_loaded`	`boolean`	`true` if YOLOv8 model loaded at startup

GET /debug — Model pipeline introspection

RESPONSE

```
{
  "yolo_model_loaded": true,
  "yolo_classes": { "0": "Grasserie", "1": "Healthy" },
  "keras_model_loaded": true,
  "keras_input_shape": "(None, 224, 224, 3)",
  "keras_output_shape": "(None, 1)",
  "pipeline": "two-stage (YOLO detect → Keras classify)",
  "fusion_weights": { "yolo": 0.4, "keras": 0.6 }
}
```

POST /predict — Silkworm disease detection ⏱ 300 req/min

REQUEST — MULTIPART/FORM-DATA

FIELD	TYPE	REQUIRED	CONSTRAINTS
`file`	`file`	Required	JPG or PNG only · max 10 MB

RESPONSE

```
{
  "detections": [
    { "class_name": "Healthy",   "confidence": 0.9142, "bbox": [120, 45, 210, 135] },
    { "class_name": "Grasserie", "confidence": 0.8734, "bbox": [340, 90, 420, 180] }
  ],
  "counts":          { "Healthy": 8, "Grasserie": 1 },
  "total":           9,
  "annotated_image": "<base64 PNG string>",
  "timestamp":       1762800000.123,
  "result":          "Grasserie",
  "confidence":      91.4
}
```

FIELD	TYPE	DESCRIPTION
`detections[].class_name`	`string`	`"Healthy"` or `"Grasserie"`
`detections[].confidence`	`float`	Fused confidence 0.0–1.0 (YOLO 40% + Keras 60%)
`detections[].bbox`	`int[4]`	Bounding box `[x1, y1, x2, y2]` in pixels
`annotated_image`	`string`	Base64 PNG — green boxes = Healthy, red = Grasserie
`result`	`string`	`"Healthy"` if no Grasserie; `"Grasserie"` if any found
`confidence`	`float`	Max individual detection confidence (0–100)

ERRORS

STATUS	CONDITION
`400`	File type not JPG/PNG
`400`	File exceeds 10 MB
`429`	Rate limit exceeded
`500`	YOLOv8 model not loaded

ℹ️ **SAHI fallback:** If ≤ 3 worms detected in the standard pass, the pipeline automatically runs sliced inference (320×320 windows, stride 160, image normalised to ≥ 1800 px wide) to handle dense tray photos.

POST `/sensor-data`

Receive IoT sensor data from ESP32 — ⏱ 120 req/min

REQUEST BODY (JSON)

```
{
  "temperature": 26.4,
  "humidity":    78.2,
  "co2":         920,
  "timestamp":   123456,
  "node_id":     "ESP32-SilkSphere",
  "zones": {
    "Zone A": { "temperature": 26.4, "humidity": 78.2, "co2": 920 }
  }
}
```

FIELD	TYPE	REQUIRED	DEFAULT
`temperature`	`float`	Required	—
`humidity`	`float`	Required	—
`co2`	`float`	Required	—
`timestamp`	`float`	Optional	null
`node_id`	`string`	Optional	`"unknown"`
`zones`	`object`	Optional	null

💾 **Side effect:** Appends a CSV row to `sensor_log.txt` as `YYYY-MM-DD HH:MM:SS, node_id, temp, humidity, co2`

GET /sensor-data/history

Historical sensor readings

QUERY PARAMETERS

PARAM	TYPE	DEFAULT	DESCRIPTION
`hours`	`int`	24	Look-back window in hours

RESPONSE

```
{
  "history": [
    { "time": "14:30", "date": "2026-11-10 14:30", "temp": 26.1,
      "humidity": 78.5, "co2": 908, "node_id": "ESP32-SilkSphere" }
  ],
  "count": 1,
  "hours": 24
}
```

GET /sensor-data/latest

Most recent reading (or smart mock)

RESPONSE

```
{
  "temperature": 26.3,
  "humidity":    77.8,
  "co2":         912,
  "light":       480,
  "nodeStatus":  "Online - Real Hardware",
  "node_id":     "ESP32-SilkSphere",
  "timestamp":   1762800000.456
}
```

ℹ️ Returns real hardware data if `sensor_log.txt` has an entry < 30 seconds old. Otherwise returns sinusoidal mock data with time-of-day light simulation (day/night cycle, peak at noon). `nodeStatus` will be `"Simulated - Smart Mock Data"` in mock mode.

GET /motor-control

Get current fan/motor command state

RESPONSE

```
{
  "motor_enabled":    false,
  "manual_override":  false,
  "timestamp":        1762800000.789
}
```

⚠️ State is **in-memory only** and resets when the backend server restarts. The ESP32 polls this endpoint to decide whether to activate the ventilation fan relay.

POST /motor-control

Set fan/motor command state

REQUEST BODY (JSON)

```
{
  "motor_enabled":   true,
  "manual_override": false
}
```

RESPONSE

```
{
  "status": "success",
  "state": {
    "motor_enabled":   true,
    "manual_override": false,
    "timestamp":       1762800001.234
  }
}
```

POST /predict-batch-state

Lifecycle stage & grade forecasting

REQUEST BODY (JSON)

```
{
  "current_stage":  "Instar 3",
  "days_in_stage":  1.5,
  "sensor_history": [
    { "temp": 26.2, "humidity": 79.0, "co2": 910.0, "timestamp": 1762800000.0 }
  ],
  "ai_health_score": 95.0
}
```

FIELD	TYPE	REQUIRED	DEFAULT
`current_stage`	`string`	Required	—
`days_in_stage`	`float`	Required	—
`sensor_history`	`array`	Required	—
`ai_health_score`	`float`	Optional	100.0

Valid `current_stage` values: `Egg`, `Instar 1` – `Instar 5`, `Spinning`, `Cocoon`

RESPONSE

```
{
  "predicted_stage_24h":    "Instar 3",
  "predicted_progress_24h": 83.3,
  "predicted_stage_48h":    "Instar 4",
  "predicted_progress_48h": 12.5,
  "expected_cocoon_grade":  "A",
  "env_compliance_score":   88.4
}
```

ℹ️ **Grade formula:** `overall = (env_compliance × 0.4) + (ai_health × 0.6)` → A ≥ 85, B ≥ 70, C ≥ 50, D < 50. Grade drops by 1 if latest reading has temp > 31°C, temp < 19°C, humidity < 55%, or CO_2 > 1300 ppm.

🗄️ Database Schema

Firestore Collections

🔥 sensor_data/latest

temperature	number
humidity	number
co2	number
light	number
nodeStatus	string
timestamp	number (ms)
zones	map | null

📡 device_commands/status

fanState	boolean
manualOverride	boolean
timestamp	number (ms)

Written by both manual user action and closed-loop automation. Read by AppContext onSnapshot listener.

📋 device_commands (collection)

command	string
type	string
timestamp	Timestamp

`command`: `"FAN_ON"` or `"FAN_OFF"`
`type`: `"manual"` or `"auto"`

📊 sensor_history (collection)

temperature	number
humidity	number
co2	number
timestamp	Timestamp

⚠️ Read by app but not yet written from the ESP32 path. Needs backend Firebase Admin write or ESP32 direct push.

sensor_log.txt (Flat File)

Located at `backend/sensor_log.txt`. Appended by every `POST /sensor-data` call.

COLUMN	FORMAT	EXAMPLE
1 — Timestamp	`YYYY-MM-DD HH:MM:SS`	`2026-11-10 14:30:22`
2 — Node ID	string	`ESP32-SilkSphere`
3 — Temperature	float	`26.4`
4 — Humidity	float	`78.2`
5 — CO_2	int	`920`
6 — Light *(optional)*	int	`480`

⚠️ No rotation or size limit. For production, replace with Firestore `sensor_history` or add a cron job to archive old data.

localStorage (Client-Side)

KEY	TYPE	CONTENTS
`ss_version`	`string`	App version `"1.0.0"`; session cleared on mismatch
`ss_session`	`JSON`	Logged-in user object (without password field)
`ss_users`	`JSON array`	All registered users including passwords

USER OBJECT SCHEMA

```
{
  "id":          "user-1762800000000",
  "name":        "Demo Farmer",
  "email":       "farmer@silk.com",
  "role":        "farmer",
  "phone":       "9876543210",
  "state":       "Karnataka",
  "farmName":    "Green Silk Farm"
}
// Buyer extra fields: companyName (instead of phone, state, farmName)
```

In-Memory State (Backend)

```
motor_control_state = {
  "motor_enabled":   False,
  "manual_override": False,
  "timestamp":       <unix float>
}
# Resets to defaults on every backend restart
```

⚠️ Migrate to Firestore `device_commands/status` for persistence across restarts.

📋 Reference

Rate Limit Summary

ENDPOINT	LIMIT	USE CASE
`POST /predict`	300 req/min	AI inference — allows real-time camera at 2 fps
`POST /sensor-data`	120 req/min	ESP32 pushes every 5 s = 12 req/min — well within limit
All other endpoints	Unlimited	—

Exceeding the limit returns HTTP `429` with: `{"error": "Rate limit exceeded: X per Y"}`

Standard Error Responses

STATUS	WHEN	BODY
`400`	Bad request (invalid file type, file too large)	`{"detail": "Invalid file type..."}`
`422`	Request body validation failure (FastAPI)	`{"detail": [{...validation errors...}]}`
`429`	Rate limit exceeded	`{"error": "Rate limit exceeded..."}`
`500`	Internal server error (model not loaded, etc.)	`{"detail": "Error processing image: ..."}`

ESP32 Firmware Data Contract

The firmware sends this exact payload to `POST /sensor-data` every 5 seconds:

```
{
  "temperature":     26.4,
  "humidity":        78.2,
  "co2":             400,          // hardcoded — real sensor not yet wired
  "timestamp":       123456,       // millis()/1000
  "node_id":         "ESP32-SilkSphere",
  "lifecycle_stage": "Instar I-III",
  "status": {
    "temperature": "OK",
    "humidity":    "ALERT",
    "co2":         "OK"
  }
}
```

⚠️ `lifecycle_stage` and `status` are sent by the firmware but are **not stored or used** by the current backend. Only `temperature`, `humidity`, `co2`, and `node_id` are persisted in `sensor_log.txt`.